

Détection de logiciel malveillant de type cheval de troie

Kamguia Kouam Eric Stephane
Département d'informatique et d'ingénierie
Université du Québec en Outaouais
Gatineau, Canada
kame10@uqo.ca

Zougrana Abdoul Fayçal
Département d'informatique et d'ingénierie
Université du Québec en Outaouais
Gatineau, Canada
zouf02@uqo.ca

Abstract— Les malwares exploitent les vulnérabilités des systèmes pour compromettre les données, souvent en se camouflant dans des applications légitimes, ce qui rend leur détection difficile avec des méthodes traditionnelles. Nous étudions dans ce projet un mécanisme de détection efficace des logiciels malveillant de type cheval de Troie. Pour ce faire, nous utilisons un jeu de données qui a été extraite de Kaggle. Il s'agit un système d'apprentissage machine que nous utiliserons pour faire cette détection des logiciels malveillants. Après une présentation et un prétraitement des données, plusieurs algorithmes d'apprentissage automatique sont testés : K-Plus Proches Voisins, arbre de décision, forêt aléatoire et régression logistique. L'analyse de leurs performances, mesurées à l'aide de métriques comme la précision et le F1-score, montre que l'arbre de décision, la forêt aléatoire et la régression logistique offrent de meilleurs résultats que les K-Plus Proches Voisins.

I. INTRODUCTION

Le développement des technologies numériques, notamment le « cloud computing », a transformé le domaine des sciences et technologies de l'information en offrant un accès à distance aux ressources et services via Internet. Cependant, cette révolution s'accompagne de nouveaux défis en matière de sécurité, avec l'augmentation des attaques de logiciels malveillants, en particulier les chevaux de Troie. Ces programmes malveillants peuvent s'infiltrer dans les systèmes, en particulier dans les environnements cloud, en exploitant des vulnérabilités pour compromettre la confidentialité, l'intégrité et la disponibilité des données. Les chevaux de Troie se dissimulent souvent dans des services ou applications légitimes, rendant leur détection complexe, surtout avec les approches traditionnelles basées sur les signatures, inefficaces face aux malwares polymorphes. Ainsi dans la suite de notre travail, nous allons étudier un jeu de données afin de pour détecter les logiciels malveillants de type cheval de Troie. Nous procéderons d'abord par une présentation de notre jeu de données. Ensuite, nous effectuerons un prétraitement des données. Puis nous ferons une analyse afin de trouver les algorithmes d'apprentissage machine qui sont adapté à l'entraînement de notre jeu de données. Enfin, nous ferons une analyse de performance pour faire un choix sur l'algorithme d'apprentissage adapté à notre étude.

I. PRESENTATION DU JEU DE DONNEES

Le jeu de donnée que nous utilisons dans notre travail a été extraite de Kaggle qui est un environnement contenant différent type de jeu de donnée orienté spécifiquement pour l'apprentissage machine. La méthode de collecte qui a été utilisé par l'auteur est lié à une supervision réseau.

Notre jeu de données est constitué de 90683 entrées de type cheval de Troie et 86799 entrées de type bienveillant. Ainsi, chaque entrée est représentée par 86 attributs dont un seul contient des données qualitatives (attribut Class) et les autres attributs sont des données quantitatives.

Nous constatons aussi que les entrées sont déjà mélangées de sorte que nous n'ayons pas uniquement toutes entrées malveillantes puis les celle bénignes.

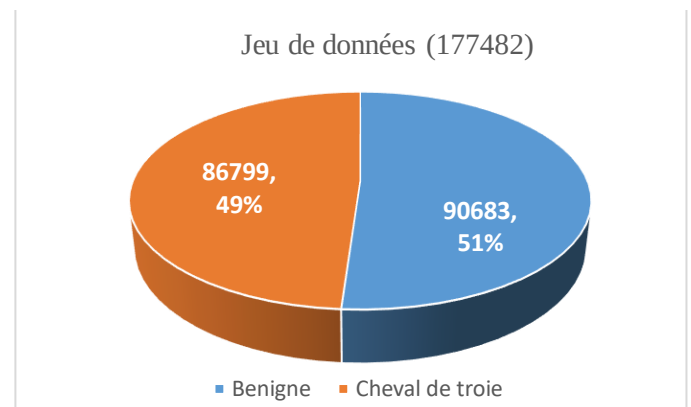


Figure1 : répartition des données

Aussi, le premier attribut ne constitue aucune information permettant de le qualifier.

II. PRETRAITEMENT

Nous allons dans la phase de prétraitement analyser le contenu nécessaire dont nous avons besoin pour effectuer notre entraînement. En effet, dans la présentation du jeu de données nous avons remarqué par exemple la présence du premier attribut ne contenant pas d'identifiant pour nous renseigner le type de valeurs que nous avons. Nous avons également constaté la présence d'attribut contenant des adresses IP et des dates. Nous effectuerons une transformation de ces attributs en numérique. Dans notre cas nous avons codé les dates afin de le jour de la semaine, le jour du mois et le mois de l'année en concerné.

```

Nombre d'attributs contenant des valeurs manquantes: 0
Attributs contenant des valeurs manquantes:
Series([], dtype: int64)
Colonnes avec des valeurs aberrantes:
Flow ID      False
Source IP    False
Destination IP False
Timestamp    False
Class        False
dtype: bool
Nombre total de colonnes avec des valeurs non numérique: 5

```

- Pour l'attribut Flow ID, nous supprimerons cet attribut car il est constitué de l'adresse IP Source, de l'adresse IP de destination, du port de destination, du port source et du protocole de chaque entrée pourtant, nous avons déjà la présence de ces différents attributs dans le jeu de donnée donc l'attribut n'apporte pas d'information supplémentaire.
- Pour la conversion numérique de l'attribut Source IP et Destination IP, nous effectuerons une transformation binaire.

Source IP	Destination
Numeric	IP Numeric

170524714	2031026004
170524714	2899904226
170524714	170524673
170524714	170524673
170524823	1796665677
170524883	170524673
170524714	1255751448
170524714	1122496352
170524714	170524673
2899905025	170524823

- Pour l'attribut Timestamp, nous la transformerons en 04 attributs à savoir le jour de la semaine, le jour du mois, le mois et le temps en minute.

day_of_week	day_of_month	month	minute
-------------	--------------	-------	--------

0	17	7	78
0	17	7	625
4	30	6	436
3	13	7	228
2	5	7	647
1	11	7	301
4	14	7	108
6	2	7	504
3	13	7	243
1	4	7	607

- Pour l'attribut class, nous effectuerons un encodage one hot.

Class_Encoded

1
1
0
1
0
1
1
0
1
1

```

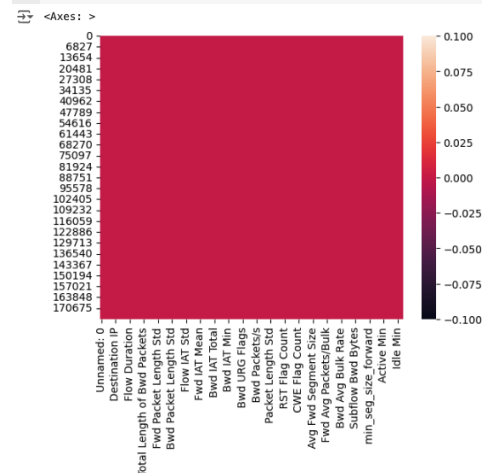
Nombre d'attributs contenant des valeurs manquantes: 0
Attributs contenant des valeurs manquantes:
Series([], dtype: int64)
Colonnes avec des valeurs aberrantes:
Series([], dtype: bool)
Nombre total de colonnes avec des valeurs non numérique: 0

```

```

#visualisation graphique des valeurs manquantes
import seaborn as sn
sn.heatmap(df.isna())

```



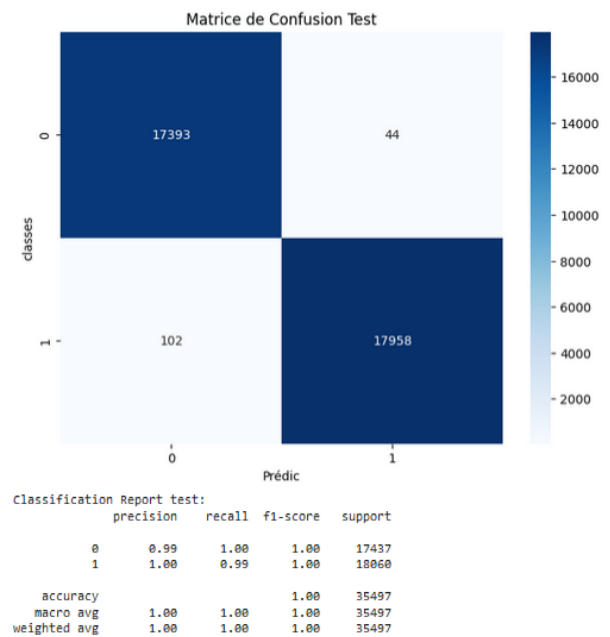
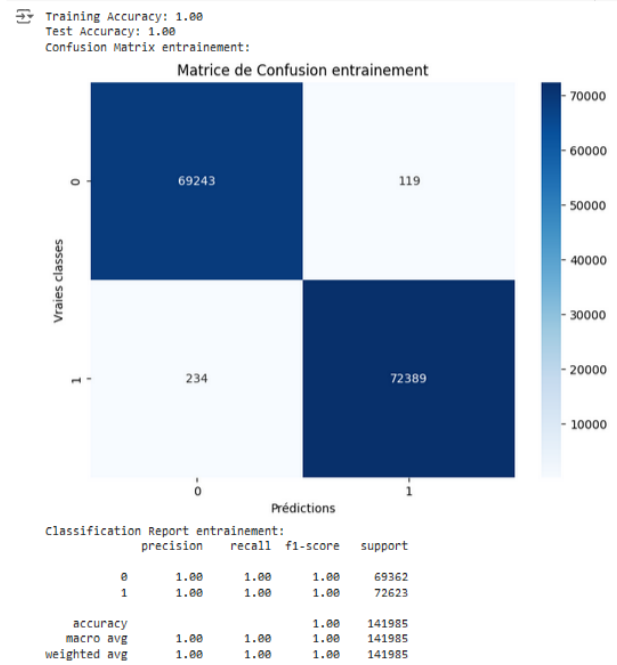
Par la suite nous avons afficher la matrice de corrélation entre les caractéristiques pour déterminer les celles qui apportent les mêmes informations afin de supprimer l'une des paires corrélées qui ont une corrélation supérieure à 0.75 pour en garder un seul.

III. ENTRAINEMENT

A présent que nous avons notre jeu de données nettoyé, nous procéderons à l'entraînement avec différents algorithmes à savoir les K plus proche voisin (KPPV), l'arbre de décision, la forêt aléatoire et la régression logistique. A la suite de l'entraînement, nous analyserons les performances afin de choisir le meilleur modèle pour notre jeu de données. Pour les différents tests que nous avons effectués, nous avons utilisé 80% pour les données d'entraînement et 20% pour les données de test. Aussi les données ont été normalisées avant l'entraînement.

- **Les K Plus Proche Voisin (KPPV)**

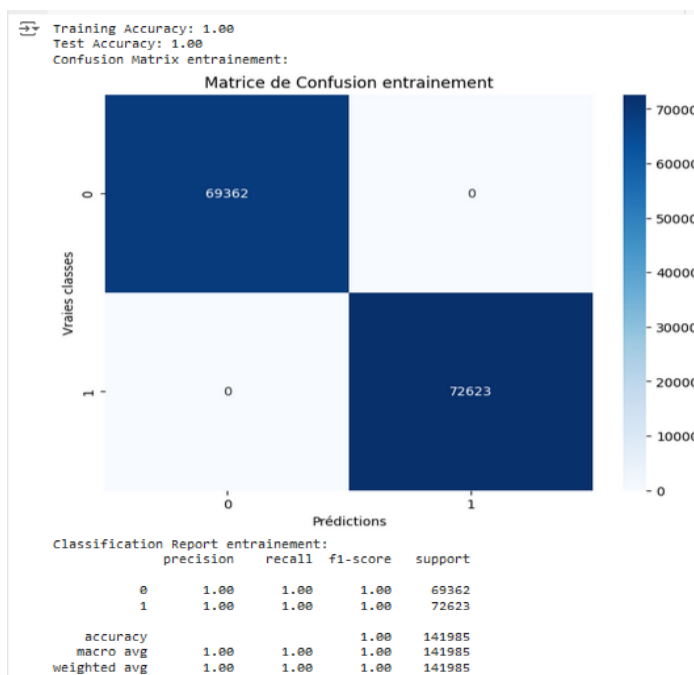
Nous avons effectué l'entraînement avec différentes valeurs de l'hyper paramètre K comme K=2 et K=5 et nous obtenons au niveau du rapport de classification des instances testées, pour la classe « 0 » à K=5, une précision de 100%, un rappel de 100% et un f1-score de 99%. Pour la classe « 1 », une précision de 100%, un rappel de 99% et un f1-score de 100%. En observant la matrice de confusion on constate une erreur de classification de 0,25% pour « 0 » et 0,0056% pour « 1 » en se basant sur la matrice de confusion.



- **L'Arbre de Décision**

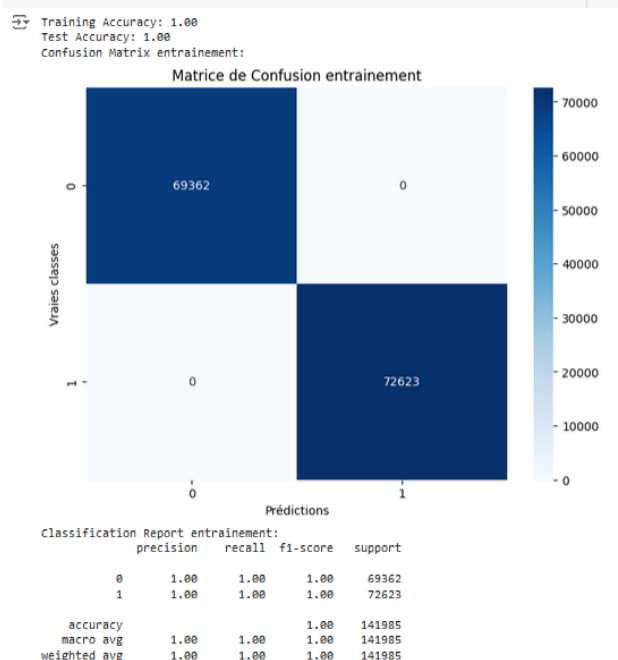
Nous avons utilisé une fonction « GridSearchCV » afin de déterminer les meilleurs hypers paramètres pour notre jeu de données.

Best parameters: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}

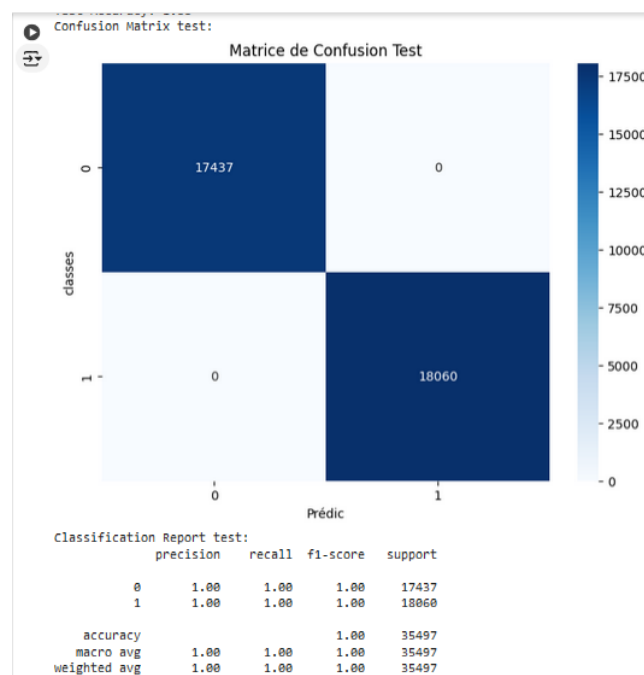
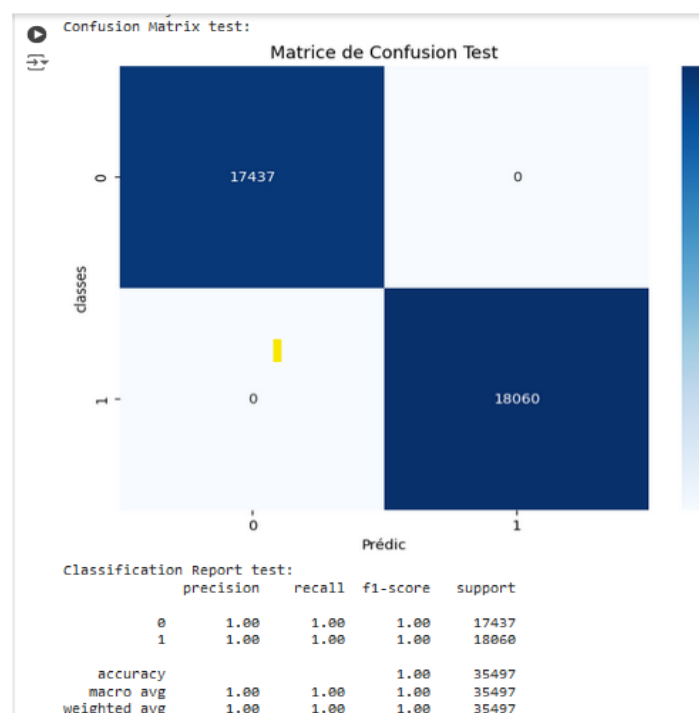


La Forêt Aléatoire

Nous avons effectué l'entraînement avec un nombre d'arbre de décision de 100 et nous avons obtenu une précision de 100% ainsi que pour le rappel et le f1-score. Aussi, la matrice de confusion nous présente un taux d'erreur de 0% pour les classes « 0 et 1 ». De plus au regard des performances de random forest sur les jeux d'entraînement et de test on peut éliminer le problème de surapprentissage ou de sous apprentissage de notre model random forest.

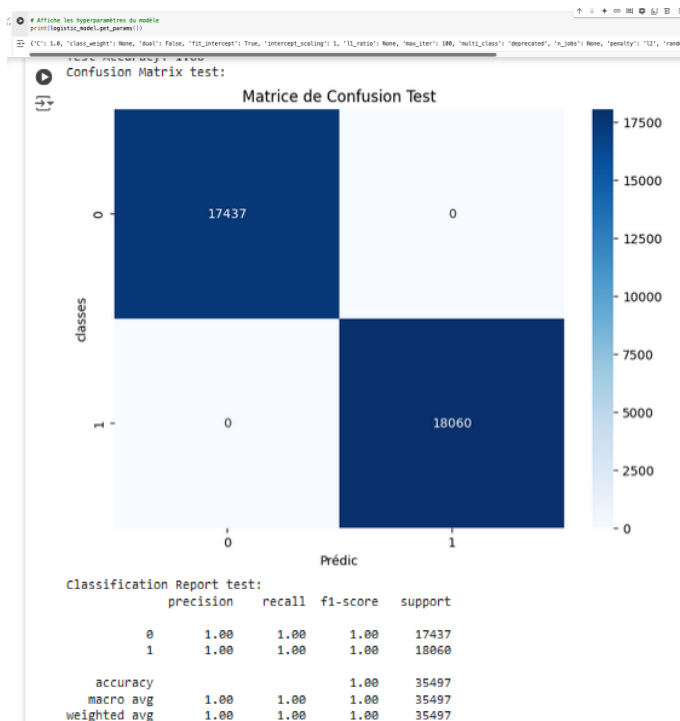
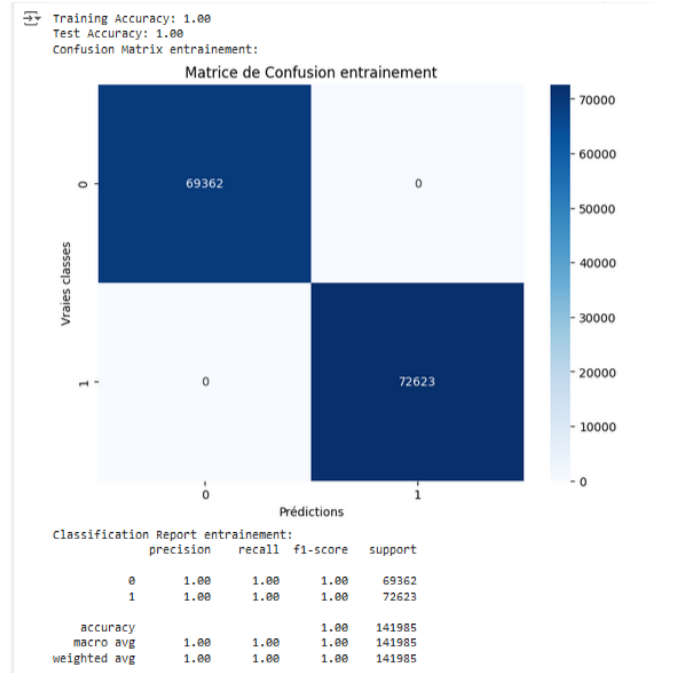


Après entraînement avec ces hyper paramètres, nous avons pu obtenir une précision de 100% ainsi que pour le rappel et le f1-score. Aussi, la matrice de confusion nous présente un taux d'erreur de 0% pour les classes « 0 et 1 »



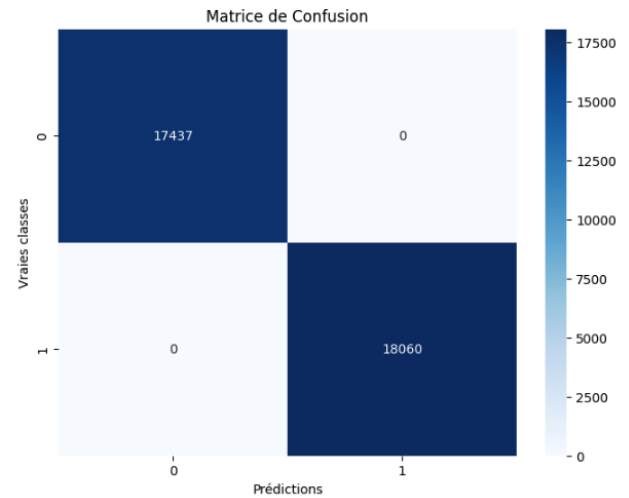
○ Le Régression Logistique

Nous avons effectué l'entraînement avec les données standardise et avons conservé les valeurs par défaut du modèle et nous obtenons une précision de 100% ainsi que le rappel et le f1-score. La matrice de confusion nous présente une classification sans erreur 0%.



Précision du modèle : 1.00
Rapport de classification :

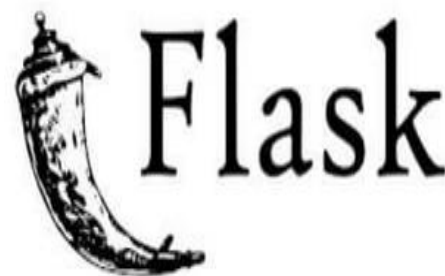
	precision	recall	f1-score	support
0	1.00	1.00	1.00	17437
1	1.00	1.00	1.00	18060
accuracy			1.00	35497
macro avg	1.00	1.00	1.00	35497
weighted avg	1.00	1.00	1.00	35497



IV. DEPLOIEMENT ET TEST DE NOTRE MODEL

Pour le déploiement de notre modèle, nous avons tout d'abord extrait de notre jeu de données deux enregistrements. Une fois notre modèle déployé, on injecte ses enregistrements et on observe le résultat.

Pour ce faire, nous avons utilisé un serveur web appelé Flask qu'on n'a déployé en local où on a intégré notre modèle random forest.



Dans un premier temps, nous avons extrait ces deux instances pour le test dans le milieu d'emploi et nous avons obtenu comme résultat :

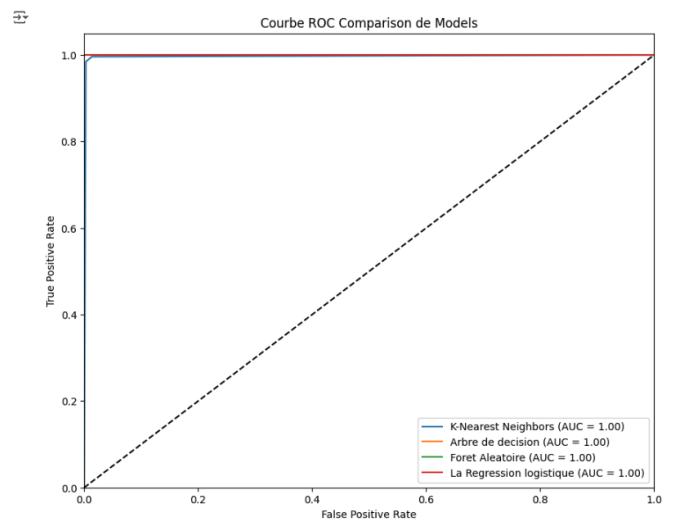
Résultats de la Détection	
Index	Prediction
0	Benign
1	Benign
Retour à l'accueil	

Alors nous nous sommes rendu compte que les instances qui ont été extraites pour le test de déploiement ne sont pas standardisées ; Ce que nous avons fait par la suite et avons obtenu comme résultat :

Résultats de la Détection	
Index	Prediction
0	Trojan
1	Benign
Retour à l'accueil	

V. ANALYSE DE PERFORMANCE

Au regard des différent test que nous avons effectué, nous avons pu constater que certains algorithmes donnent de meilleurs résultats comparativement aux autres. Pour être plus explicite, nous avons les algorithmes arbre de décision, la forêt aléatoire et la régression logistique qui nous donnent une précision de 100% ainsi que le rappel et le f1-score. Aussi, la matrice de confusion présente aucune erreur de classifications pour ces trois algorithmes. Nous pouvons donc déduire que ces algorithmes effectuent une meilleure classification comparativement à l'algorithmes KPPV.



CONCLUSION

Nous avons travaillé sur la classification d'un jeu de données de détection de logiciel malveillant de type cheval de Troie. Nous avons commencé par effectuer un prétraitement adéquat afin de s'assurer d'avoir de bonnes données prêtes à être entraînées. Ainsi, nous avons effectué l'entraînement du jeu de données avec 04 algorithmes à savoir Les K Plus proches voisins, l'arbre de décision, la forêt aléatoire ainsi que la régression logistique. Nous avons pu constater au regard des performances de chacun des algorithmes (la précision, le rappel, le f1-score et la matrice de confusion) que l'arbre de décision, la forêt aléatoire et la régression linéaire donnent de meilleur résultat comparativement aux K Plus Proches Voisins. Comme possible amélioration qu'on pourrais apporter dans un futur proche dans le cadre de ce travail nous pourrions utiliser la technique qui combine le suréchantillonnage et le sous échantillonnage pour équilibrer notre jeu de données.

REFERENCES

- [1] https://scikit-learn.org/stable/supervised_learning.html
- [2] Kanaker, H., Abdel Karim, N., Awwad, S. A. B., Ismail, N. H. A., Zraqou, J., & Al Ali, A. M. F. Trojan Horse Infection Detection in Cloud-Based Environment Using Machine Learning. International Journal of Interactive Mobile Technologies, 2024
- [3] Kamalakanta Sethi, Shankar Kumar Chaudhary. Indian Institute of Technology Bhubaneswar. A Novel Malware Analysis for Malware Detection and Classification using Machine Learning Algorithms
- [4] <https://www.kaspersky.fr/resource-center/threats/trojans>, Consulté le 20/11/2024 à 12 :24 pm